

PreciseDose:

Application Guide

1. Introduction and Platform Overview

The PreciseDose platform is an integrated environment for modeling and simulating drug delivery, designed on the principles of pharmacokinetics (PK) and pharmacodynamics (PD). By employing advanced computational algorithms, the system enables precise prediction of drug behavior within the body while accounting for individual physiological parameters.

- Unique Capabilities
- Multistage simulation of drug absorption and distribution pathways
- Real-time computation of critical pharmacokinetic parameters
- Organ-level distribution prediction across defined time intervals
- Integrated development environment for customized and advanced analyses



Figure 1. *View of the app's home page.*



2. Model Validation

Comprehensive Validation Approach

The modeling framework was developed using a reverse-engineering approach, meaning the drug delivery design was validated inversely from organ absorption back to the initial dose.

Validation was conducted using both external simulation tools and reference pharmacokinetic datasets to ensure the model's reliability and physiological plausibility. Specifically, the Teoreler web-based pharmacokinetic modeling platform developed by the University of Liverpool and widely used in PBPK and long-acting therapeutic projects was employed for comparative validation.

The model was trained on the dataset Bictegravir – Adult (Oral, 100 mg, Subramanian et al., 2023) and tested using Risperidone – Oral data. Drug- and subject-specific parameters were entered into Teoreler, and predicted plasma concentration–time profiles were compared with those generated by our model. The results showed consistent and physiologically reasonable alignment, with only minor deviations attributable to numerical rounding and parameter granularity.

Model outputs were further cross-validated against pharmacokinetic data reported in authoritative references, including PubChem, DrugBank, FDA Pharmacokinetic Reviews, and standard pharmacology texts such as Goodman & Gilman's The Pharmacological Basis of Therapeutics and Clinical Pharmacokinetics (Rowland & Tozer). Predicted PK parameters (e.g., C_max, T_max, AUC) fell within reported physiological and pharmacological ranges, confirming the model's robustness and predictive reliability.

3. System Architecture and Input Parameters

- Hierarchical Model Structure
- Base Inputs → Diffusion Calculations → Local Absorption → Metabolism → Organ Distribution
- Input Parameter Categories



a) Demographic Parameters

- Physiological characteristics (weight, height, age, sex)
- Body mass and metabolic indices

b) Drug Properties

- Physicochemical characteristics (lipophilicity, molecular radius)
- Formulation parameters (dose, release profile)

c) Personalization Factors

- Individual modifiers (scale: 0.5–2.0)
- Nanoformulation factors (scale: 1–10)
- Absorption efficiency parameters

4. Integrated Development Environment (Code Editor)

To provide an environment for intelligent parameter estimation and customized analytical modeling without leaving the application interface. Users can export structured data outputs directly through built-in Python and pandas functions within the Code Editor tab.

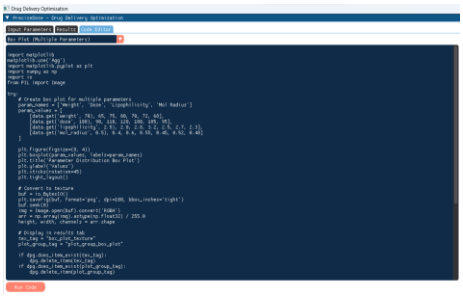


Figure 2. *View of the editor's code tab.*



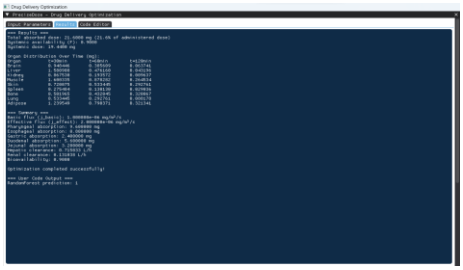


Figure 3. *Display the results of drug distribution calculations and editor code numerically.*

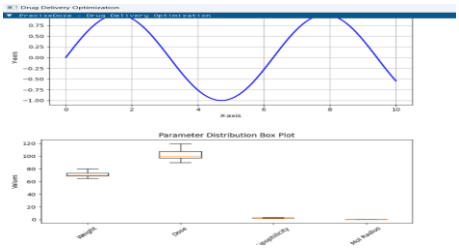


Figure 4. *Simple graphic of the user's analysis of the original output via the editor's code tab.*

Available Libraries

```
# Advanced statistical analysis
import numpy as np
import scipy.stats
import pandas as pd

# Machine learning and AI
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR
import torch.nn as nn

# Professional visualization
import matplotlib.pyplot as plt
```



```
import seaborn as sns

import plotly.graph_objects as go
```

Accessible Data Dictionary

(Patient parameters, drug parameters, physiological values, modifiers, and calculated outcomes are structured as shown below.)

Example:

```
data['age'] # Age (years)

data['weight'] # Weight (kg)

data['height'] # Height (cm)

data['gender'] # Gender (0: Female, 1: Male)

...

results['organ_distribution'] # Dictionary of organ-level distribution
```

Each organ entry contains proportional distribution and kinetic constants, e.g.:

```
organ_data['Liver'] = {'percent': 27, 'k': 0.04}
```

5. Advanced Analytical Strategies

- Implemented Algorithms
- Parameter sensitivity analysis
- Multi-objective dose optimization
- Monte Carlo simulations for uncertainty assessment
- Nonlinear regression for curve fitting



Specialized Modules

```
def estimate_volume_of_distribution(logP, molecular_weight):
    """Estimate distribution volume based on lipophilicity"""
    return optimized_Vd_calculation(logP, molecular_weight)

def predict_half_life(clearance, Vd):
    """Predict half-life based on clearance and distribution volume"""
    return (0.693 * Vd) / clearance
```

6. Internal Validation Protocols

- Automated Quality Control
- Validation of input parameter ranges
- Dimensional consistency checks
- Convergence verification for numerical algorithms
- Real-Time Monitoring Module
- Computational error logging
- Resource consumption tracking
- Comprehensive process auditing

7. System Limitations and Development Strategies

Design Constraints

1. Core Code Protection

- Encrypted execution of the main code
- Access limited through defined APIs



2. Coding Requirements

- Strict adherence to Python indentation rules
- Standardized module import structure
- Development Solutions
- Use of pre-designed templates
- Integration of standardized data dictionaries
- Incremental testing of customized modules

8. Outputs and Reporting

- Automated Reports
- Computed pharmacokinetic parameters
- Organ-level distribution plots
- Parameter sensitivity analyses
- Concentration–time predictions

Output Formats

- Structured textual reports
- High-resolution visual charts
- Exportable raw data files

Execution Instructions

The main application file (PreciseDose.py) is encrypted and must be executed within a Python-supported IDE. Ensure that the complete project folder including required images and libraries is opened to prevent technical or graphical issues.



An additional module, qua.py, simulates the user's Quantum Digital Twin. To use it, extract necessary parameters from the main app, input them into qua.py, and execute.

If import warnings for the encrypted core file appear, they can safely be ignored.

Quantum-Biological Digital Twin System (qua.py)

Overview and Purpose

An advanced quantum-biological modeling system that, using outputs from PreciseDose, generates interactive 3D simulations of drug distribution across body organs.

Required Inputs

- Output parameters from PreciseDose
- Systemic dose and bioavailability
- Time-dependent organ distribution
- Clearance and metabolic parameters

System Outputs

- Quantum 3D distribution maps
- Interactive spider charts for comparative analysis
- Dynamic heat maps of concentration variations
- Time-series plots with quantum effects
- Quantum phase-space dynamics

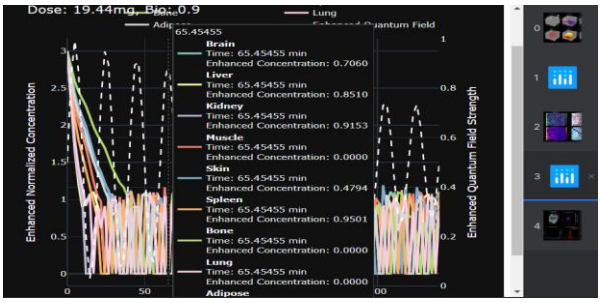


Figure 5. *An example of an interactive graph of a digital twin plugin.*



Key Advantages

- Quantum-based modeling incorporating tunneling and bioenergy field effects
- Interactive 3D visualization with dynamic rendering
- Adaptive sensitivity to individualized inputs
- Comprehensive quantitative and qualitative reporting

Applications

- Preclinical drug distribution research
- Personalized medicine and therapy prediction
- Drug development and formulation optimization

Execution Workflow

1. Extract data from PreciseDose

2. Input parameters into qua.py

3. Run the simulation and generate outputs

4. Analyze results and export reports



This module serves as an advanced complement to PreciseDose, providing deeper insight into drug behavior at the quantum-biological level.

Note: The quantum modeling component is based on theoretically validated frameworks. Outputs are constrained within physiologically reasonable limits. Dedicated experimental validation is planned for future versions. The current release of qua.py is intended primarily for educational and conceptual exploration.

The PreciseDose platform, through the implementation of state-of-the-art pharmacokinetic modeling techniques, offers a fully professional environment for researchers, pharmacists, and medical scientists working in personalized medicine. With its robust data analytics and integrated development environment, it represents a powerful tool for optimizing therapeutic regimens.

Acknowledgment

The development of this platform is the result of months of specialized research in pharmacokinetic modeling. Expert feedback from professionals like you plays a crucial role in improving its quality and functionality.

We Welcome Your Expert Feedback

As collaborative researchers, we are eager to receive your analytical insights, edge case observations, and development suggestions.

Bug Reporting and Optimization

If you encounter:

Discrepancies between outputs and experimental data

Unexpected behavior under specific parameter conditions

Identified development needs

Please contact the development team directly at: askarizade16@gmail.com